spring-bazaar

要件定義書

v0.4 | Draft | 2026年4月

cartId ヘッダー・PENDING 用途・エラーレスポンス形式を明記

プロジェクト名	spring-bazaar
バージョン	0.4（改版）
作成日	2026年4月
ステータス	Draft
スコープ	商品管理・カート機能（認証は Phase 2）
改版内容	cartId ヘッダー方針・PENDING 用途・エラーレスポンス形式を明記

1. プロジェクト概要

spring-bazaar は Java（Spring Boot）と React（TypeScript）を用いたフルスタック学習プロジェクトです。ECサイトのコアドメイン（商品閲覧・カート・モック注文）を通じて、Spring Boot の REST API 設計・JPA によるデータ永続化・レイヤードアーキテクチャの実践的な理解を目的とします。

2. スコープ

2-1. 対象（In Scope）

- 商品一覧・詳細の閲覧（ページネーション対応）
- カートへの商品追加・変更・削除
- 注文確定（在庫減算あり・決済はモック）
- 管理者による商品登録（POST のみ）

2-2. 対象外（Out of Scope）

- ユーザー認証・JWT 管理 → Phase 2
- 商品の更新（PUT）・削除（DELETE） → Phase 2
- 決済処理の実連携 → Phase 2
- 商品画像のファイルアップロード → Phase 2

3. 機能要件

商品一覧表示	商品をカード形式で一覧表示。ページネーション対応（Page<T>）	High
商品詳細表示	商品名・価格・説明・在庫数を表示	High
カートへの追加	商品詳細からカートに追加。数量指定・在庫チェックあり	High
カート確認	カート内商品の一覧・合計金額を表示	High
カートの変更	数量変更・商品削除が可能	Medium
注文確定（モック）	カート内容をOrderにスナップショット保存・在庫を減算	Medium
管理者：商品登録	商品名・価格・在庫数・画像 URL を登録（POST のみ）	Low

4. 非機能要件

パフォーマンス

- 商品一覧 API のレスポンスタイム：500ms 以内（ローカル環境）

セキュリティ

- CORS 設定：フロントエンド（localhost:5173）のみ許可
- SQL インジェクション対策：JPA パラメータバインディングを使用

バリデーション

- spring-boot-starter-validation を使用（@Valid / @NotBlank / @Min 等）
- DTO に Bean Validation アノテーションを付与し Controller 層で @Valid を適用
- 在庫チェック（quantity <= stock）は Service 層でドメインロジックとして実装

保守性

- レイヤードアーキテクチャ（domain / application / presentation / infrastructure）に従う
- springdoc-openapi で Swagger UI を自動生成
- config/ はファイル単位で分割（SecurityConfig / CorsConfig / BeanConfig）

5. API 設計（概要）

ベース URL：http://localhost:8080

カート系エンドポイントは全て X-Cart-Id: {uuid} リクエストヘッダーで cartId を渡す。フロントは localStorage の値をヘッダーにセットする。

GET	/api/products	商品一覧（ページネーション）	Page<ProductResponse> ★
GET	/api/products/{id}	商品詳細取得	ProductResponse
POST	/api/products	商品登録（管理者用）★追加	ProductResponse
GET	/api/cart	カート内容取得	CartResponse
POST	/api/cart/items	カートに商品追加	CartResponse
PATCH	/api/cart/items/{id}	カート商品の数量変更	CartResponse
DELETE	/api/cart/items/{id}	カートから商品削除	204 No Content
POST	/api/orders	注文確定（モック）	OrderResponse

★ 商品一覧は Page<T> でページネーション対応。クエリパラメータ: ?page=0&size=20&sort=name,asc

6. バリデーション方針

spring-boot-starter-validation を使用。@Valid を Controller の @RequestBody に付与し、違反時は @RestControllerAdvice で 400 Bad Request を返す。

POST /api/products	name	@NotBlank — 商品名は必須
POST /api/products	price	@DecimalMin("0.01") — 0より大きい値
POST /api/products	stock	@Min(0) — 0以上の整数
POST /api/cart/items	productId	@NotNull — 商品ID必須
POST /api/cart/items	quantity	@Min(1) — 1以上。在庫数以下かはService層でチェック
PATCH /api/cart/items/{id}	quantity	@Min(1) — 1以上

7. エラーレスポンス形式

全エラーは @RestControllerAdvice で統一フォーマットに変換して返す。フロントはこの形式を前提に型定義・エラーハンドリングを実装する。

レスポンスボディ（共通フォーマット）

```
{ "status": 400, "error": "Bad Request", "message": "name は必須です", "timestamp": "2026-04-13T10:00:00Z" }
```

400	Bad Request	バリデーションエラー（@Valid 違反）。message にフィールドエラー詳細

404	Not Found	ProductNotFoundException など。対象リソースが存在しない
409	Conflict	OutOfStockException。在庫不足で注文確定できない場合
500	Internal Server Error	予期しないサーバーエラー

8. データモデル

Phase 1 の設計方針：JPA エンティティ = ドメインモデルとして同一クラスで扱う。規模拡大時に分離を検討する。

Product

- id: UUID（PK）
- name: String — @NotBlank
- description: String
- price: BigDecimal — @DecimalMin("0.01")
- stock: Integer — @Min(0)
- createdAt / updatedAt: LocalDateTime

Cart / CartItem

- cartId: UUID — フロントの localStorage で生成・保持。サーバーはそのまま受け取る
- productId: UUID（FK → Product）
- quantity: Integer — @Min(1)

カート識別の方針：認証が存在しない Phase 1 では、フロント側で crypto.randomUUID() を用いて UUID を生成し localStorage に保存する。サーバー側セッション管理は不要。

Order / OrderItem

- orderId: UUID（PK）
- status: Enum（CONFIRMED / PENDING） — Phase 1 は即 CONFIRMED。PENDING は Phase 2 の決済連携時に使用予定
- totalPrice: BigDecimal — 注文確定時点の金額スナップショット
- OrderItem.unitPrice: BigDecimal — 注文確定時点の単価スナップショット

注文確定のライフサイクル

在庫チェック → 在庫減算 → Cart のスナップショットを OrderItem として保存 → Cart をクリア → status: CONFIRMED を返す

- ① Service 層で Cart の全 CartItem を走査し在庫チェック（不足時は OutOfStockException）
- ② Product.stock を減算して保存（楽観的ロックは Phase 2 以降）
- ③ CartItem の内容（productId / quantity / 現時点の price）を OrderItem にコピー（スナップショット）
- ④ Cart をクリア
- ⑤ Order を status=CONFIRMED で保存して OrderResponse を返す

9. アーキテクチャ設計

9-1. バックエンド：レイヤー構成

domain/model/	Entity (JPA)	JPAアノテーション付きのドメインモデル。Phase 1 はEntity=ドメインモデル
domain/repository/	Repository I/F	Spring Data JPA の interface のみ
domain/exception/	ドメイン例外	ProductNotFoundException / OutOfStockException など
application/service/	UseCase ロジック	トランザクション境界。ドメインモデルを操作し DTO を返す
application/dto/	Input / Output DTO	外部公開データ形式。@Valid アノテーションを付与
application/mapper/	Entity↔DTO 変換	手動変換メソッドを集約（Phase 1）

`presentation/controller/`	**REST Controller**	@RestController。リクエスト受付と DTO の返却のみ
`presentation/advice/`	**例外ハンドラ**	@RestControllerAdvice。例外→HTTP レスポンス変換
`infrastructure/config/`	**設定クラス**	SecurityConfig / CorsConfig / BeanConfig をファイル分割
`infrastructure/persistence/`	**Repository 実装**	domain の interface を実装

9-2. フロントエンド：ディレクトリ構成

- src/api/ — axios クライアント・BaseURL・interceptor
- src/types/ — バックエンド DTO に対応する TypeScript 型
- src/components/ — 汎用 UI パーツ（ドメイン非依存）
- src/features/ — 商品 / カート をドメイン別に分割。各 feature が components / hooks / api を内包
- src/hooks/ — feature 横断の共通カスタム hooks
- src/pages/ — React Router v7 のページコンポーネント

9-3. 状態管理方針

サーバー状態（商品・カート）は TanStack Query で管理。グローバル UI 状態が必要になった場合は Zustand を追加する。

- 商品データ → TanStack Query（キャッシュ・再フェッチ）
- カートデータ → TanStack Query（cartId を localStorage から取得してリクエスト）
- グローバル UI 状態 → 必要になった時点で Zustand を導入

10. テスト方針

Unit Test	`application/service/`	Mockito でリポジトリをモック。ビジネスロジック・例外を検証
@WebMvcTest	`presentation/controller/`	MockMvc でHTTPリクエスト/レスポンスを検証。Service はモック
@DataJpaTest	`infrastructure/persistence/`	H2インメモリDBでクエリ・CRUD動作を検証
Integration Test	任意（`Phase 2` 以降）	@SpringBootTest で全層を通した結合テスト

Phase 1 の目標：Service 層の Unit Test と Controller 層の @WebMvcTest を最優先で実装する。@DataJpaTest はカスタムクエリを書いた場合に追加する。

11. 技術スタック

バックエンド

- Java 21 / Spring Boot 3.x
- Spring Data JPA（Hibernate）
- spring-boot-starter-validation
- Spring Security（Phase 1 は CORS 設定のみ）
- Gradle（Kotlin DSL）
- PostgreSQL 16
- springdoc-openapi（Swagger UI）

フロントエンド

- React 19 / TypeScript / Vite
- TanStack Query（サーバー状態管理）
- React Router v7
- axios（HTTP クライアント）
- Zustand（グローバル UI 状態 / 必要時に追加）

開発環境

- macOS（M4 Pro）/ IntelliJ IDEA / VS Code
- PostgreSQL：Homebrew でローカル起動
- Docker：Phase 2 以降で導入予定

12. フェーズ計画

Phase 1（現フェーズ）

- Spring Initializr でプロジェクト生成
- Product CRUD API（GET / GET{id} / POST）+ ページネーション
- バリデーション（@Valid）+ 例外ハンドリング（@RestControllerAdvice）
- Cart API（取得・追加・変更・削除）
- 注文確定 API（在庫減算・スナップショット保存）
- React フロントエンド（商品一覧・詳細・カート画面）
- cartId を localStorage で管理
- Service 層 Unit Test + Controller 層 @WebMvcTest

Phase 2（予定）

- JWT 認証の導入（Spring Security + JJWT）
- ユーザー登録・ログイン画面
- 商品の PUT / DELETE エンドポイント
- 楽観的ロック（@Version）による在庫競合対策
- application/mapper/ を MapStruct に置き換え
- Docker Compose によるコンテナ化
- CI/CD（GitHub Actions）